

```

        nElements = len(self.dataList)
        return nElements

    ⑥ def show(self):
        # Show the stack in a vertical orientation
        print('Stack is:')
        for value in reversed(self.dataList):
            print('    ', value)

```

Listing 8-8: A stack as a Python class

The `Stack` class keeps track of all the data using a list instance variable named `self.dataList` ①. The client doesn't need to know this level of detail, but `push()` ② just adds an item to the internal list using the Python `append()` operation, while `pop()` ③ pops the last element from the internal list. Because it is easy to do, this implementation of the `Stack` class also implements three additional methods:

- `peek()` ④ allows the caller to obtain the data at the top of the stack without removing it from the stack.
- `getSize()` ⑤ returns the number of items on the stack.
- `show()` ⑥ prints the contents of the stack in the way that the client thinks of a stack: the data is displayed vertically, with the most recent data pushed shown at the top. This can be helpful in debugging client code that involves multiple calls to `push()` and `pop()`.

This was an extremely simple example, but as you gain more experience writing classes, your classes will typically become more complex. Along the way, you may find cleaner and more efficient ways of writing some methods and perhaps rewrite them. Because objects provide both encapsulation and abstraction, as the writer of a class, you should feel free to modify its code and data, as long as the published interfaces do not change. Changes to the implementation of methods should have no ill effects on client software but rather should allow you to make improvements without affecting any client code. In fact, if you find ways to make your code more efficient and publish a new version, client code may appear to speed up, with zero changes required to that code.